



Experiment 08

TCP Socket Programming in Python

Name: Deven

UID: 24BCF10060

Aim

To implement a simple client-server communication model using TCP (Transmission Control Protocol) sockets in Python, and to understand the connection-oriented nature of TCP communication.

Learning Objectives

1. Understand what a socket is and the role of Python's `socket` module.
2. Understand the connection-oriented nature of TCP and the three-way handshake.
3. Create a TCP server socket, bind it to an address, and listen for connections.
4. Create a TCP client socket and connect it to the server.
5. Exchange messages between client and server using `send()/recv()` over an established connection.
6. Observe and verify TCP communication in real time.

Theory

What is a Socket?

A *socket* is a software endpoint that lets two programs communicate over a network. It is identified by an IP address and a port number, and Python exposes it through the built-in `socket` module.

TCP: Connection-Oriented Communication

TCP (Transmission Control Protocol) is a *connection-oriented*, reliable, ordered byte-stream protocol. Before any data can be exchanged, the client and server must establish a connection through a **three-way handshake**: the client sends a **SYN**, the server replies with **SYN-ACK**, and the client confirms with an **ACK**. Python's `socket` calls hide the individual handshake packets, but the handshake still happens at the network level whenever `connect()` is called.

Key Socket Calls Used

- `socket.socket(AF_INET, SOCK_STREAM)` – creates an IPv4 TCP socket (`SOCK_STREAM` means a reliable, connection-oriented byte stream).
- `bind((host, port))` – attaches the server socket to a specific address and port.
- `listen(n)` – marks the socket as passive/listening, with a backlog of `n` pending connections.
- `accept()` – blocks until a client connects, then returns a *new* socket dedicated to that client plus its address.
- `connect((host, port))` – actively establishes a connection to the server, triggering the three-way handshake.

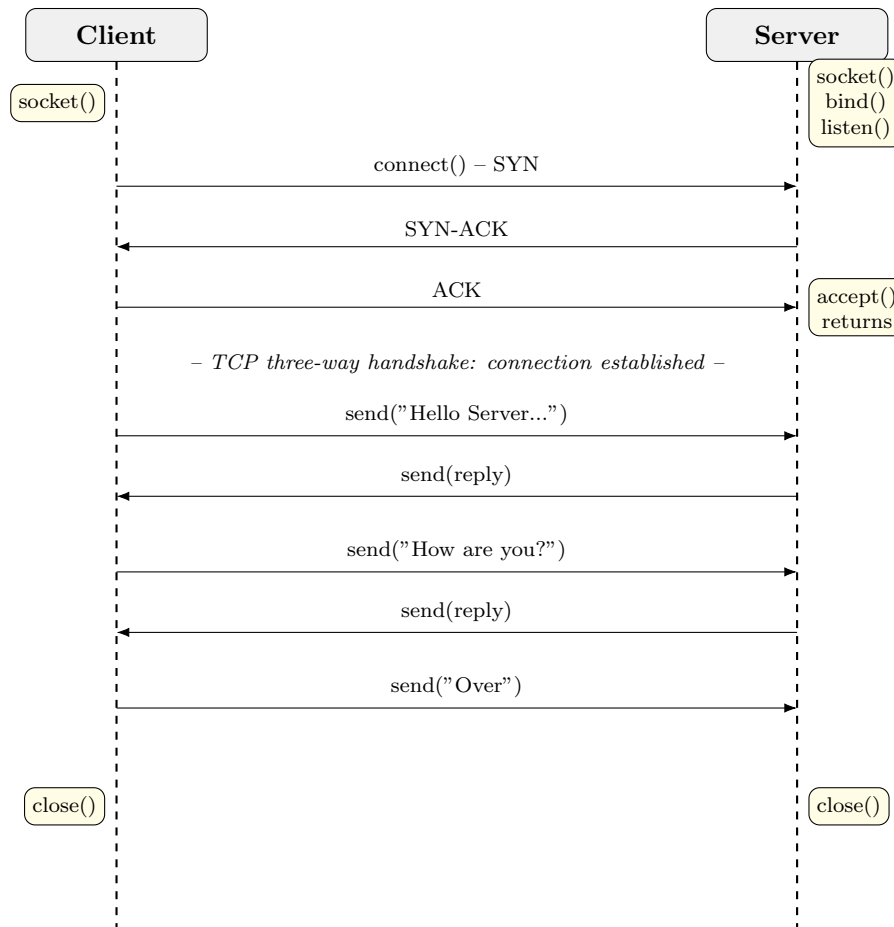


- `send()` / `recv()` – exchange data over the already-established, reliable, ordered connection.
- `close()` – terminates the connection.

Because TCP is stream-based and connection-oriented, data always arrives reliably and in order, but only after a connection has been explicitly opened (`connect()`/`accept()`) and it must be explicitly closed (`close()`) afterwards.

Communication Sequence

The diagram below shows the exact exchange implemented in this experiment: the three-way handshake, followed by two request/reply exchanges, followed by the client signalling the end of the session with "Over".





Procedure

Part A – server.py

```
1 import socket
2
3 server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
4 ## AF_INET: use IPv4. SOCK_STREAM: use TCP.
5
6 server_socket.bind(("127.0.0.1", 5000))    # 127.0.0.1 = localhost
7 server_socket.listen(1)                    # start waiting for
      connections
8
9 print("Server started")
10 print("Waiting for a client...")
11
12 client_socket, client_address = server_socket.accept()
13 ## accept() blocks here until a client connects.
14
15 print("Client connected:", client_address)
16
17 while True:
18     message = client_socket.recv(1024).decode()
19     print("Client:", message)
20
21     if message == "Over":
22         break
23
24     reply = input("Enter reply: ")
25     client_socket.send(reply.encode())
26
27 client_socket.close()
28 server_socket.close()
29 print("Connection closed")
```



Part B – client.py

```
1 import socket
2
3 client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
4
5 client_socket.connect(("127.0.0.1", 5000))
6 ## Client actively connects to the server -> triggers the 3-way
7 handshake.
8
9 print("Connected to server")
10
11 while True:
12     message = input("Enter message: ")
13     client_socket.send(message.encode())    # encode(): string ->
14     bytes
15
16     if message == "Over":
17         break
18
19     reply = client_socket.recv(1024).decode()
20     print("Server:", reply)
21
22 client_socket.close()
23 print("Connection closed")
```

Part C – Running Order

The server must be started *first*, since it blocks on `accept()` waiting for a connection. Once the server is running and waiting, the client is started, which connects, and the two exchange messages interactively until the client sends "Over".



Verification / Output

Server terminal

```
Server started
Waiting for a client...
Client connected: ('127.0.0.1', 56338)
Client: Hello Server, this is client
Enter reply: Hi Client, message received!
Client: How are you?
Enter reply: I am fine, thanks for asking!
Client: Over
Connection closed
```

Note: the client's port number (56338 above) is assigned automatically by the OS and will differ on every run.

Client terminal

```
Connected to server
Enter message: Hello Server, this is client
Server: Hi Client, message received!
Enter message: How are you?
Server: I am fine, thanks for asking!
Enter message: Over
Connection closed
```

Result

A TCP connection was successfully established between a Python client and server on 127.0.0.1 port 5000 using the three-way handshake (`connect()`/`accept()`). Two rounds of messages were exchanged reliably over the persistent connection using `send()`/`recv()`, and the connection was cleanly closed by both sides after the client sent "Over" – confirming the connection-oriented behaviour of TCP.

Learning Outcome

After completing this experiment, the following learning outcomes were achieved.

- Understood the role of a socket as a network communication endpoint.
- Understood TCP's connection-oriented model and its three-way handshake (SYN, SYN-ACK, ACK).
- Implemented a TCP server using `bind()`, `listen()`, and `accept()`.
- Implemented a TCP client using `connect()`, and exchanged data using `send()`/`recv()`.
- Verified reliable, ordered, two-way communication over a single persistent TCP connection.